GRUPO 4 · REQUERIMIENTOS FUNCIONALES

Transferencias y Pagos

Módulo: Transferencias, beneficiarios, importes y estados

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 4
Versión	1.0
Fecha	10 de octubre de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST, `/api/v2`) y aiquaa-sandbox-web (front, `/v2`)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training_v2
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre la agenda de beneficiarios de un cliente (alta, consulta, modificación y baja) y las transferencias de dinero desde una cuenta: internas, hacia otra cuenta del sandbox, o externas, hacia un beneficiario agendado de otro banco. La transferencia se ejecuta en el momento —debita el origen, acredita el destino si es interna y registra los movimientos— y puede anularse con un contraasiento.

Fuera de alcance: El pago de facturas y servicios (en este curso "pagos" son las transferencias a beneficiarios); las transferencias programadas o recurrentes; la conversión de moneda; comisiones e impuestos; los límites por operación o por día (solo existen en la pantalla web); y la liquidación real hacia otro banco: una transferencia externa debita el origen y no acredita ninguna cuenta.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con la API key de su grupo (o con un token JWT de grupo) y automatiza escenarios BDD sobre el módulo de su grupo.
Cliente del banco (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es titular de cuentas, tarjetas, préstamos, beneficiarios, planes de ahorro y depósitos.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo sobre el schema `qa_training_v2`, bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz tipo home banking que consume la API a través del proxy `/api/proxy/v2/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v2/beneficiarios	RF-G4-01	Lista beneficiarios vigentes, opcionalmente por cliente.
POST /api/v2/beneficiarios	RF-G4-02	Agenda un beneficiario para un cliente.
GET /api/v2/beneficiarios/{id}	RF-G4-03	Devuelve el detalle de un beneficiario.
PUT /api/v2/beneficiarios/{id}	RF-G4-04	Reemplaza los datos de un beneficiario.
DELETE /api/v2/beneficiarios/{id}	RF-G4-05	Da de baja un beneficiario.
GET /api/v2/transferencias	RF-G4-06	Lista transferencias, por cuenta de origen y/o estado.
POST /api/v2/transferencias	RF-G4-07	Ejecuta una transferencia interna o a un beneficiario.
GET /api/v2/transferencias/{id}	RF-G4-08	Devuelve el detalle de una transferencia.
POST /api/v2/transferencias/{id}/anular	RF-G4-09	Anula una transferencia completada y revierte el dinero.
POST /api/v2/g4/auth/token	RF-G4-TK	Emite el token JWT del grupo.

2. Precondiciones y acceso

- **Base URL de la API:** `https://aiquaa-sandbox-api.vercel.app`. La documentación interactiva del curso está en `/docs/v2` y la especificación OpenAPI en `GET /api/v2/docs`. La aplicación web está en `https://aiquaa-sandbox-web.vercel.app`.
- Toda request debe autenticarse por una de dos vías: el header `x-api-key` con una clave activa de `public.api_keys`, o el header `Authorization: Bearer <token>` con un JWT de grupo vigente. Si se envían ambos, manda `x-api-key`. Sin credenciales, o con credenciales inválidas, inactivas o vencidas, la respuesta es `401 UNAUTHORIZED` con un mensaje genérico que no indica cuál fue el motivo.
- Las rutas bajo `/api/v2` solo aceptan credenciales del **curso 2**: una API key válida pero del curso 1 recibe `403 FORBIDDEN` con el mensaje "Esta API key pertenece al curso 1 y esta ruta es del curso 2.". El gate es simétrico: una key del curso 2 tampoco sirve en `/api/v1`.
- Cada credencial admite un máximo de **30 requests por minuto** (ventana deslizante, contada por API key o por usuario del token, no por IP). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.
- Los datos de práctica viven en el schema aislado `qa_training_v2` (10 tablas) y se cargan con `scripts/seed-data-v2.sql`: 8 clientes (más un cliente por grupo, usado por el login de token), 12 cuentas, 20 movimientos, 10 tarjetas, 6 préstamos con sus cuotas, 10 beneficiarios, 12 transferencias, 6 planes de ahorro y 6 depósitos. La data inicial es determinística, pero **el sandbox es compartido por los cinco grupos**: los registros sembrados pueden haber sido modificados por otra suite. Un escenario robusto crea sus propios datos en lugar de depender del estado de una fila sembrada.
- **Clientes de soporte:** `GET|POST /api/v2/usuarios` y `GET|PUT|DELETE /api/v2/usuarios/{id}` no pertenecen a ningún grupo y están disponibles para todos; sirven para crear el titular de los productos de cada escenario. `GET /usuarios` acepta el filtro `email`; `POST` exige `nombre`, `email` y `documentoNumero` (email y documento son únicos: repetirlos da `409 CONFLICT`).
- **Sandbox SQL:** `POST /api/v2/sql/select` y `POST /api/v2/sql/update` reciben `{ "sql", "params" }` y permiten consultar o preparar datos directamente sobre las 10 tablas del curso. Admiten un único statement por request, con placeholders `$1..$n`; el `UPDATE` exige cláusula `WHERE`. Son la vía para verificar en base lo que hizo un endpoint, o para forzar un estado que la API no produce.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` hace baja lógica (`activo = false`, o `activa = false` en `cuentas`) y devuelve `204 No Content` sin cuerpo. Una fila dada de baja deja de verse en `GET`, `PUT` y `DELETE`, pero sigue existiendo en la base.
- En la aplicación web el acceso tiene tres capas, en orden: elegir el curso (`/curso`, opción 2), cargar la API key del curso (`/login`, que valida que la key sea del curso elegido) y elegir el cliente con el que se opera (`/auth/login`, por email de un cliente activo). Las pantallas `/v2/usuarios/*` son la excepción a la tercera capa: se pueden usar solo con la API key, porque son el punto de partida para crear un cliente.
- Para transferir se necesita una cuenta de origen `activa` con saldo suficiente. Una cuenta recién abierta nace en cero: hay que fondearla con `POST /api/v2/cuentas/{id}/movimientos` (tipo `credito`) antes de transferir.
- Una transferencia interna requiere además una cuenta de destino `activa` y en la misma moneda que el origen; una externa requiere un beneficiario vigente.

- Los endpoints de cuentas (`/api/v2/cuentas/**`) pertenecen al Grupo 1, pero este módulo los usa para preparar datos y para verificar saldos y movimientos después de transferir.
- Datos sembrados: 10 beneficiarios (el 8 está dado de baja) y 12 transferencias con los cuatro estados representados: `completada`, `pendiente` (5 y 11), `rechazada` (7) y `anulada` (9). Las cuentas 2, 7 y 12 son en dólares; la cuenta 8 está `bloqueada` y la 10 `cerrada`.

3. Modelo de datos del módulo

beneficiarios

Agenda de destinos externos de cada cliente: cuentas de terceros en otros bancos.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`: el cliente dueño de la agenda
`nombre`	`text`, obligatorio: titular de la cuenta destino
`banco`	`text`, obligatorio; texto libre
`numero_cuenta`	`text`, obligatorio; único por cliente (`usuario_id` + `numero_cuenta`)
`alias`	`text`, opcional
`activo`	`boolean`, obligatorio, por defecto `true`; `false` = dado de baja
`created_at`	`timestamptz`, por defecto `now()`

transferencias

Transferencias ejecutadas. Cada una tiene exactamente un destino: una cuenta interna o un beneficiario, nunca ambos ni ninguno.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`cuenta_origen_id`	`bigint`, obligatorio, referencia a `cuentas(id)`
`cuenta_destino_id`	`bigint`, opcional, referencia a `cuentas(id)`; debe ser distinta de la de origen
`beneficiario_id`	`bigint`, opcional, referencia a `beneficiarios(id)`
`monto`	`numeric(14,2)`, obligatorio, mayor que cero
`moneda`	`text`, `PYG` o `USD`; la API copia la moneda de la cuenta de origen
`concepto`	`text`, opcional
`referencia`	`text`, obligatoria y única; la genera la API
`estado`	`text`, uno de `pendiente` / `completada` / `rechazada` / `anulada`; por defecto `completada`
`activo`	`boolean`, por defecto `true`; ningún endpoint lo cambia
`created_at`	`timestamptz`, por defecto `now()`

Una transferencia mueve dinero de verdad dentro del sandbox: después de ejecutarla, el saldo del origen baja y —si es interna— el del destino sube, y cada cuenta afectada recibe un movimiento con `referencia_tipo = 'transferencia'` y `referencia_id` igual al id de la transferencia. La referencia generada tiene la forma `TRF-` + fecha y hora con milisegundos + `-` + id de la cuenta de origen; las sembradas usan otro formato (`TRF-2026-0001`).

4. Requerimientos funcionales

RF-G4-01 — Listar beneficiarios

`GET /api/v2/beneficiarios`

Devuelve los beneficiarios vigentes, con la posibilidad de acotar a la agenda de un cliente.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.

Reglas de negocio

- Devuelve únicamente beneficiarios no dados de baja (`activo = true`).
- Sin `usuarioId` devuelve beneficiarios de todos los clientes.
- Un cliente sin beneficiarios, o un `usuarioId` inexistente, devuelve una lista vacía y no un error.
- Máximo 100 filas, ordenadas por identificador ascendente.

Respuesta esperada

- `200 OK` con `data` como arreglo de beneficiarios; cada elemento incluye `id`, `usuario_id`, `nombre`, `banco`, `numero_cuenta`, `alias`, `activo` y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
VALIDATION_ERROR	400	`usuarioId` no es un entero positivo.

Fuente: `app/api/v2/beneficiarios/route.ts`

RF-G4-02 — Agendar un beneficiario

`POST /api/v2/beneficiarios`

Agrega a la agenda de un cliente una cuenta de otro banco a la que podrá transferir.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo del cliente dueño de la agenda.
- `nombre` (obligatorio): texto no vacío.
- `banco` (obligatorio): texto no vacío.
- `numeroCuenta` (obligatorio): texto no vacío.
- `alias` (opcional): texto libre.

Reglas de negocio

- Un mismo cliente no puede agendar dos veces el mismo número de cuenta: responde 409 `CONFLICT`. Dos clientes distintos sí pueden agendar el mismo número.
- La unicidad incluye a los beneficiarios dados de baja: volver a agendar un número que ese cliente tuvo y dio de baja también responde 409.
- El cliente debe existir; un `usuarioId` inexistente es rechazado por la clave foránea.
- **No se valida el formato del número de cuenta ni que el banco exista**: ambos son texto libre. El alias no es único.

Respuesta esperada

- `201 Created` con `data` conteniendo el beneficiario creado.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, está vacío, o el `usuarioId` no existe.
`CONFLICT`	409	Ese cliente ya tiene (o tuvo) un beneficiario con el mismo número de cuenta.

Fuente: `app/api/v2/beneficiarios/route.ts`

RF-G4-03 — Consultar un beneficiario

`GET /api/v2/beneficiarios/{id}`

Devuelve el detalle de un beneficiario puntual.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Si no existe o está dado de baja, responde 404 con el mensaje "Beneficiario no encontrado.".

Respuesta esperada

- `200 OK` con `data` conteniendo el beneficiario completo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un beneficiario vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/beneficiarios/[id]/route.ts`

RF-G4-04 — Modificar un beneficiario

`PUT /api/v2/beneficiarios/{id}`

Reemplaza nombre, banco, número de cuenta y alias de un beneficiario.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `nombre`, `banco` y `numeroCuenta` (obligatorios): textos no vacíos.
- `alias` (opcional): texto libre.

Reglas de negocio

- Es un reemplazo: omitir `alias` lo deja nulo, aunque antes tuviera valor.
- El dueño de la agenda (`usuario_id`) no se puede cambiar.
- Cambiar el número de cuenta por uno que ese cliente ya tiene agendado responde 409 `CONFLICT`.
- Modificar un beneficiario no altera las transferencias ya hechas hacia él.

Respuesta esperada

- `200 OK` con `data` conteniendo el beneficiario ya actualizado.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un beneficiario vigente con ese identificador.
`CONFLICT`	409	El nuevo número de cuenta ya está agendado por ese cliente.
`VALIDATION_ERROR`	400	Falta un campo obligatorio o está vacío.

Fuente: `app/api/v2/beneficiarios/[id]/route.ts`

RF-G4-05 — Dar de baja un beneficiario

`DELETE /api/v2/beneficiarios/{id}`

Quita un beneficiario de la agenda mediante baja lógica.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Marca `activo = false`; la fila permanece en la base.
- No verifica transferencias asociadas: un beneficiario con transferencias se puede dar de baja, y esas transferencias siguen consultables.
- Un beneficiario dado de baja ya no puede usarse como destino de una transferencia nueva.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe un beneficiario vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/beneficiarios/[id]/route.ts`

RF-G4-06 — Listar transferencias

`GET /api/v2/transferencias`

Devuelve las transferencias del sandbox, con la posibilidad de acotar por cuenta de origen y por estado.

Entradas y validaciones

- `cuentaOrigenId` (opcional, por query): número entero positivo.
- `estado` (opcional, por query): `pendiente`, `completada`, `rechazada` o `anulada`.

Reglas de negocio

- Los dos filtros se combinan entre sí.
- No hay filtro por cuenta de destino, por beneficiario, por cliente ni por fecha.
- Las transferencias anuladas siguen apareciendo, con `estado = 'anulada'`: anular no es dar de baja.
- Máximo 100 filas, ordenadas por identificador ascendente.

Respuesta esperada

- `200 OK` con `data` como arreglo de transferencias; cada elemento incluye `id`, `cuenta_origen_id`, `cuenta_destino_id`, `beneficiario_id`, `monto`, `moneda`, `concepto`, `referencia`, `estado`, `activo` y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`cuentaOrigenId` no es un entero positivo, o `estado` no es uno de los cuatro valores permitidos.

Fuente: `app/api/v2/transferencias/route.ts`

RF-G4-07 — Ejecutar una transferencia

`POST /api/v2/transferencias`

Transfiere un importe desde una cuenta hacia otra cuenta del sandbox (interna) o hacia un beneficiario agendado (externa). La transferencia se ejecuta en el acto y queda `completada`.

Entradas y validaciones

- `cuentaOrigenId` (obligatorio): número entero positivo.
- `cuentaDestinoId` (opcional): número entero positivo. Indica una transferencia interna.
- `beneficiarioId` (opcional): número entero positivo. Indica una transferencia externa.
- `monto` (obligatorio): número mayor que cero, como número JSON.
- `concepto` (opcional): texto libre.

Reglas de negocio

- Debe indicarse **exactamente uno** de `cuentaDestinoId` y `beneficiarioId`. Enviar ambos, o ninguno, responde 400 con el mensaje "Indicá exactamente uno: cuentaDestinoId (interna) o beneficiarioId (externa).".
- La cuenta de destino no puede ser la de origen: responde 400 con el mensaje "La cuenta destino no puede ser la misma que la de origen.".
- La cuenta de origen debe existir y no estar dada de baja (si no, 404 "Cuenta origen no encontrada."), y estar en estado `activa` (si no, 409 "La cuenta origen está bloqueada: no admite transferencias.").
- El saldo del origen debe alcanzar: si no, 409 "Saldo insuficiente en la cuenta origen.". Transferir exactamente el saldo es válido.
- En una transferencia interna, la cuenta de destino debe existir (404 "Cuenta destino no encontrada."), estar `activa` (409 "La cuenta destino está bloqueada: no admite acreditaciones.") y tener la misma moneda que el origen (409 "Las cuentas deben tener la misma moneda.").
- En una transferencia externa, el beneficiario debe existir y estar vigente (404 "Beneficiario no encontrado.").
- Las validaciones se evalúan en ese orden: la primera que falla determina la respuesta.
- Si todo es válido: se debita el origen, se acredita el destino (solo si es interna), se crea la transferencia con `estado = 'completada'`, la moneda del origen y una `referencia` generada, y se registra un movimiento `debito` en el origen y —si es interna— uno `credito` en el destino.
- La descripción de esos movimientos es el `concepto`; si no se envió, es "Transferencia enviada" y "Transferencia recibida".
- La operación es atómica: ante cualquier rechazo no cambia ningún saldo ni se registra nada.
- **No se valida que las cuentas pertenezcan al mismo cliente, ni que el beneficiario sea del titular de la cuenta de origen.**

Respuesta esperada

- `201 Created` con `data` conteniendo la transferencia creada y, anidadas, `data.cuenta_origen` (con su saldo ya debitado) y `data.cuenta_destino` (con su saldo ya acreditado, o `null` si la transferencia es externa).

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta `cuentaOrigenId` o `monto`, el monto no es un número positivo, se indicaron los dos destinos o ninguno, o el destino es la misma cuenta de origen.
`NOT_FOUND`	404	No existe la cuenta de origen, la cuenta de destino o el beneficiario.
`CONFLICT`	409	Alguna cuenta no está activa, el saldo no alcanza, o las monedas difieren.

Fuente: `app/api/v2/transferencias/route.ts`

RF-G4-08 — Consultar una transferencia `GET /api/v2/transferencias/{id}`

Devuelve el comprobante de una transferencia: importe, cuentas, referencia y estado.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Si no existe, responde 404 con el mensaje "Transferencia no encontrada.".
- La respuesta no incluye las cuentas anidadas: solo los identificadores.
- Una transferencia no se puede modificar ni dar de baja: `PUT` y `DELETE` sobre esta ruta responden 405. Para revertirla existe la anulación (RF-G4-09).

Respuesta esperada

- `200 OK` con `data` conteniendo la transferencia completa.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una transferencia con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v2/transferencias/[id]/route.ts`

RF-G4-09 — Anular una transferencia

`POST /api/v2/transferencias/{id}/anular`

Revierte una transferencia completada: devuelve el dinero al origen, lo retira del destino si fue interna, registra los movimientos inversos y deja la transferencia en estado `anulada`.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `motivo` (opcional, en el cuerpo): texto libre. Se usa como descripción de los movimientos de reversa.

Reglas de negocio

- Solo se anula una transferencia en estado `completada`. En otro estado responde 409 con el mensaje "Solo se puede anular una transferencia completada (está 'anulada')." (con el estado que corresponda).
- Si fue interna, la cuenta de destino debe conservar saldo suficiente para devolver el importe: si ya lo gastó, responde 409 con el mensaje "La cuenta destino no tiene saldo suficiente para revertir la transferencia." y nada cambia.
- Si fue interna, se debita el destino y se registra en él un movimiento `debito`; en todos los casos se acredita el origen y se registra en él un movimiento `credito`. Ambos llevan `referencia_tipo = 'transferencia'` y el id de la transferencia.
- La descripción de esos movimientos es el `motivo`; si no se envió, es "Anulación de transferencia " seguido de la referencia.
- La transferencia no se borra: conserva sus datos y pasa a `estado = 'anulada'`. Anular es definitivo: no existe la operación inversa.
- **No se valida el estado actual de las cuentas**: una transferencia se puede anular aunque el origen o el destino estén hoy `bloqueada` o `cerrada`.
- No hay plazo máximo para anular.

Respuesta esperada

- `200 OK` con `data` conteniendo la transferencia con `estado = 'anulada'` y, en `data.cuenta_origen`, la cuenta de origen con el saldo ya restituido. La cuenta de destino no viene en la respuesta.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una transferencia con ese identificador.
`CONFLICT`	409	La transferencia no está completada, o el destino ya no tiene saldo para devolver.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo, o `motivo` no es texto.

Fuente: `app/api/v2/transferencias/[id]/anular/route.ts`

RF-G4-TK — Obtener el token del grupo

`POST /api/v2/g4/auth/token`

Intercambia el usuario y la contraseña del grupo por un token JWT que puede usarse en lugar de la API key en cualquier ruta del curso. La ruta en sí también exige estar autenticado (API key del curso 2 o un token vigente).

Entradas y validaciones

- `username` (obligatorio): texto de 1 a 64 caracteres. El usuario del grupo es `c2_g04_transferencias`.
- `password` (obligatorio): texto de 1 a 200 caracteres. Se entrega por el canal del curso; no se publica en este documento.

Reglas de negocio

- La contraseña se compara en la base de datos contra un hash bcrypt; nunca se devuelve ni se registra en claro.
- Usuario inexistente, contraseña incorrecta y credencial inactiva devuelven el mismo `400 VALIDATION_ERROR` con el mensaje "Usuario o password inválidos.": la respuesta no revela cuál de los tres falló. Es `400` y no `401` a propósito, para no confundirlo con el fallo de la API key.
- Si las credenciales son válidas pero pertenecen a otro grupo, la respuesta es `403 FORBIDDEN` con el mensaje "Estas credenciales pertenecen al grupo 5 y esta ruta es del grupo 4.".
- El token es un JWT firmado con HS256, emitido por `aiquaa-sandbox`, con vigencia de **3600 segundos**. Sus claims (`sub`, `usuarioId`, `grupo`, `curso`) son legibles: el token está firmado, no cifrado.
- El token identifica al grupo pero no restringe por grupo: con él se puede operar sobre los recursos de cualquier módulo del curso 2.
- La emisión no deja rastro en tablas de dominio (el curso 2 no tiene tabla de sesiones); solo queda la fila de auditoría.

Respuesta esperada

- `200 OK` con `data` conteniendo `token`, `tokenType` (`Bearer`), `expiresIn` (`3600`), `grupo` (`4`), `grupoNombre`, `curso` (`2`) y `usuario` (`id`, `nombre`, `email`, `username`).

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta `username` o `password`, están vacíos, o la combinación no es válida.
`FORBIDDEN`	403	Las credenciales son de otro grupo, o la API key usada para llamar es del curso 1.
`UNAUTHORIZED`	401	La request no trae API key ni token.

Fuente: `app/api/v2/g4/auth/token/route.ts`, `lib/group-token-route.ts` y `lib/jwt.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de cohorte (curso) → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Tipos en la respuesta:** los identificadores (`id`, `usuario_id`, `cuenta_id`, ...) viajan como **número**; los importes y tasas (`saldo`, `monto`, `limite_credito`, `tasa_anual`, ...) viajan como **texto con dos decimales** (por ejemplo `"5000000.00"`), por ser valores numéricos exactos de la base; las fechas sin hora viajan como `YYYY-MM-DD` y las marcas de tiempo (`created_at`, `fecha_pago`) en formato ISO 8601. Una aserción sobre un importe debe comparar contra texto o convertir a número antes.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo). Los campos no declarados en el esquema se ignoran sin error.
- **Coerción de tipos:** los identificadores (`id`, `usuarioId`, `cuentaId`, ...) se convierten de texto a número automáticamente, vengan por query, por ruta o en el cuerpo. Los **importes, tasas y plazos del cuerpo JSON deben enviarse como número**: enviarlos como texto (`"monto": "1000"`) devuelve `400 VALIDATION_ERROR`.
- **Cuerpo JSON:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body."; un cuerpo vacío se interpreta como `{}`. Un fallo del esquema devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid request." y el detalle del campo en `error.details`.
- **Reglas de negocio → 409:** cuando el cuerpo es válido pero el estado actual del recurso no permite la operación (saldo insuficiente, recurso en un estado que no admite la acción), la respuesta es `409 CONFLICT` con un mensaje en español que describe la regla. `400` queda para entradas inválidas.
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`, por ejemplo un `usuarioId` inexistente) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos (una fecha imposible, un número fuera de rango) cae en `400 EXECUTION_ERROR`. En los tres casos el `message` es el texto crudo de Postgres.
- **Importes con más de dos decimales:** la API no valida la cantidad de decimales; la base redondea a dos al guardar.
- **Operaciones que mueven dinero:** se ejecutan en una transacción con bloqueo de las filas involucradas; si alguna validación falla, no queda ningún cambio parcial (ni saldo modificado ni movimiento registrado).
- **Listados:** devuelven como máximo 100 filas ordenadas por identificador ascendente, sin paginación ni indicación de que hubo recorte. Los filtros opcionales se combinan entre sí (`AND`).
- **Headers de caudal:** toda respuesta que pasó el control de caudal incluye `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset` (este último en milisegundos Unix).
- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera el framework y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** las requests quedan registradas en `public.sql_audit_log` con la credencial usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Sin control de titularidad:** la API no tiene roles ni noción de "mis" productos. Cualquier credencial válida del curso puede leer y modificar los recursos de cualquier cliente, y ningún endpoint verifica que la cuenta indicada pertenezca al `usuarioId` indicado.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso, y las acciones que cambian un estado sin crear un registro (bloquear, activar, aprobar, pagar una cuota, anular, cancelar).
`201 Created`	POST que crea un recurso o registra un movimiento de dinero nuevo.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Faltan credenciales, la API key no existe o está inactiva, o el token es inválido o venció.
`FORBIDDEN`	403	La credencial es válida pero es de otro curso, o las credenciales de un grupo se usaron en la ruta de token de otro grupo.
`RATE_LIMITED`	429	Se superaron las 30 requests por minuto de esa credencial.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe o ya está dado de baja.
`CONFLICT`	409	Regla de negocio violada sobre un recurso que existe, o violación de una restricción `UNIQUE`.
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/v2/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú muestra los módulos del grupo del alumno (más "Clientes", siempre visible) cuando su email figura en el roster del curso; si no figura, muestra todos. Cada listado arranca filtrado por el cliente con el que se inició sesión y permite cambiar a otro titular o a "todos"; la búsqueda, el orden y la paginación de los listados se resuelven en el navegador. **El front es deliberadamente más estricto que la API en varios puntos**: esas reglas están marcadas como "regla de UI" y solo se pueden probar por pantalla — la misma operación enviada directo a la API es aceptada. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local.

Pantalla	Qué permite hacer
`/v2/beneficiarios` — Agenda de beneficiarios	Tabla con alias o nombre, banco, número de cuenta y cliente; filtro por titular y búsqueda por nombre, alias, banco o número, con un acceso directo "Transferir" por fila. Ejecuta RF-G4-01.
`/v2/beneficiarios/new` y `/v2/beneficiarios/{id}`	Alta de un beneficiario (RF-G4-02) y detalle con edición (RF-G4-04), baja (RF-G4-05) y botón para transferirle.
`/v2/transferencias` — Listado de transferencias	Tabla con referencia, origen, destino, monto, fecha y estado; filtros por titular y estado, y búsqueda por referencia o concepto. Ejecuta RF-G4-06; como la API solo filtra por cuenta de origen, el filtro por titular se resuelve en el navegador cruzando con las cuentas del cliente.
`/v2/transferencias/new` — Nueva transferencia	Asistente de cuatro pasos: "Destino" (cliente, entre mis cuentas o a un beneficiario, cuenta de origen y destino), "Monto" (importe y concepto, con el saldo y el tope diario restante a la vista), "Confirmación" (resumen y saldo final proyectado) y "Comprobante". Ejecuta RF-G4-07.
`/v2/transferencias/{id}` — Comprobante	Muestra referencia, estado, monto, fecha, origen y destino, y permite anular con un motivo (RF-G4-09). Ejecuta RF-G4-08.

- **Regla de UI:** tope diario por cuenta de origen de 50.000.000 de guaraníes o 10.000 dólares, calculado sumando las transferencias `completada` del día. La API no tiene ningún límite.
- **Regla de UI:** una transferencia interna solo se puede hacer entre cuentas activas del mismo cliente ("Entre mis cuentas"). La API permite transferir a la cuenta de cualquier cliente.
- **Regla de UI:** el concepto admite hasta 60 caracteres. La API no limita su longitud.
- **Regla de UI:** para anular, el motivo es obligatorio y debe tener entre 10 y 120 caracteres. La API lo acepta opcional y sin límites.
- **Regla de UI:** el número de cuenta de un beneficiario debe tener entre 6 y 20 dígitos, el nombre entre 3 y 120 caracteres, el alias entre 3 y 30 y no repetirse dentro de la agenda del cliente, y el banco se elige de una lista de bancos de Paraguay. La API solo exige que nombre, banco y número no estén vacíos.
- El importe mayor al saldo o al tope diario se rechaza en el navegador antes de llegar a la API; si la API rechaza la operación, el asistente vuelve al paso del campo observado.
- El botón "Anular transferencia" solo aparece en transferencias `completada`.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G4-01 — Listar beneficiarios

- Al listar sin filtro, la respuesta es 200 y ningún elemento tiene `activo` en falso.
- Al filtrar por un cliente, todos los elementos devueltos tienen ese `usuario_id`.
- Al filtrar por un cliente sin beneficiarios, la respuesta es 200 con un arreglo vacío.
- Un beneficiario dado de baja con RF-G4-05 deja de aparecer en el listado.

RF-G4-02 — Agendar un beneficiario

- Al agendar un beneficiario válido, la respuesta es 201 y `data.activo` es verdadero.
- Al agendar sin `alias`, la respuesta es 201 y `data.alias` es nulo.
- Al agendar por segunda vez el mismo `numeroCuenta` para el mismo cliente, la respuesta es 409 `CONFLICT`.
- Al agendar el mismo `numeroCuenta` para otro cliente, la respuesta es 201.
- Al agendar con `nombre` vacío o sin `banco`, la respuesta es 400 `VALIDATION_ERROR`.
- Al agendar para un `usuarioId` inexistente, la respuesta es 400 `VALIDATION_ERROR`.

RF-G4-03 — Consultar un beneficiario

- Dado un beneficiario recién agendado, al consultarlo la respuesta es 200 y sus datos coinciden con los enviados.
- Al consultar un identificador inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Beneficiario no encontrado.".
- Al consultar el beneficiario sembrado 8 (dado de baja), la respuesta es 404 `NOT_FOUND`.

RF-G4-04 — Modificar un beneficiario

- Dado un beneficiario existente, al cambiar su `banco` la respuesta es 200 y el cambio se refleja al volver a consultarlo.
- Al modificar sin enviar `alias`, la respuesta es 200 y `data.alias` queda nulo.
- Al cambiar el `numeroCuenta` por el de otro beneficiario del mismo cliente, la respuesta es 409 `CONFLICT`.
- Al modificar un beneficiario inexistente o dado de baja, la respuesta es 404 `NOT_FOUND`.

RF-G4-05 — Dar de baja un beneficiario

- Dado un beneficiario vigente, al darlo de baja la respuesta es 204 sin cuerpo.
- Tras la baja, RF-G4-03 responde 404 y el beneficiario deja de aparecer en RF-G4-01.
- Tras la baja, una transferencia hacia ese beneficiario responde 404 `NOT_FOUND` con el mensaje "Beneficiario no encontrado.".
- Dar de baja dos veces el mismo beneficiario devuelve 204 la primera vez y 404 la segunda.

RF-G4-06 — Listar transferencias

- Al filtrar por `cuentaOrigenId`, todos los elementos devueltos tienen esa cuenta como origen.
- Al filtrar por `estado=anulada`, todos los elementos devueltos tienen `estado` igual a `anulada`.
- En cada elemento, exactamente uno de `cuenta_destino_id` y `beneficiario_id` tiene valor y el otro es nulo.
- Una transferencia recién ejecutada aparece al filtrar por su cuenta de origen.
- Al enviar `estado=procesada`, la respuesta es 400 `VALIDATION_ERROR`.

RF-G4-07 — Ejecutar una transferencia

- Dadas dos cuentas activas en guaraníes y un origen con saldo 1.000.000, al transferir 300.000 la respuesta es 201, `data.estado` es `completada`, `data.cuenta_origen.saldo` es `"700000.00"` y `data.cuenta_destino.saldo` aumentó en 300.000.

- Después de una transferencia interna, el extracto de cada cuenta tiene un movimiento nuevo con `referencia_tipo` igual a `transferencia` y `referencia_id` igual al `id` de la transferencia: `debito` en el origen y `credito` en el destino.
- Dado un beneficiario vigente, al transferirle 100.000 la respuesta es 201, `data.beneficiario_id` tiene valor, `data.cuenta_destino_id` es nulo y `data.cuenta_destino` es `null`.
- La `referencia` devuelta comienza con `TRF-` y la `moneda` coincide con la de la cuenta de origen.
- Al transferir exactamente el saldo disponible, la respuesta es 201 y el origen queda en `"0.00"`.
- Al transferir un importe mayor al saldo, la respuesta es 409 `CONFLICT` con el mensaje "Saldo insuficiente en la cuenta origen." y ningún saldo cambia.
- Al enviar `cuentaDestinoId` y `beneficiarioId` a la vez, o ninguno de los dos, la respuesta es 400 `VALIDATION_ERROR`.
- Al transferir de una cuenta a sí misma, la respuesta es 400 `VALIDATION_ERROR`.
- Al transferir de una cuenta en guaraníes a una en dólares, la respuesta es 409 `CONFLICT` con el mensaje "Las cuentas deben tener la misma moneda.".
- Al transferir desde o hacia una cuenta `bloqueada`, la respuesta es 409 `CONFLICT`.
- Al transferir hacia una cuenta o un beneficiario inexistente, la respuesta es 404 `NOT_FOUND` y el saldo del origen no cambia.
- Al enviar `monto` igual a 0, negativo o como texto, la respuesta es 400 `VALIDATION_ERROR`.

RF-G4-08 — Consultar una transferencia

- Dada una transferencia recién ejecutada, al consultarla la respuesta es 200 y su `monto`, cuentas y `referencia` coinciden con los devueltos al crearla.
- Al consultar una transferencia inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Transferencia no encontrada.".
- Al enviar `PUT` o `DELETE` sobre una transferencia existente, la respuesta es 405.

RF-G4-09 — Anular una transferencia

- Dada una transferencia interna recién ejecutada por 300.000, al anularla la respuesta es 200, `data.estado` es `anulada` y los saldos de origen y destino vuelven a los valores previos a la transferencia.
- Tras la anulación, el extracto del origen tiene un movimiento `credito` y el del destino un movimiento `debito`, ambos por 300.000 y referidos a la transferencia.
- Dada una transferencia externa recién ejecutada, al anularla el saldo del origen vuelve a su valor previo.
- Al anular enviando un `motivo`, ese texto aparece como `descripcion` de los movimientos de reversa.
- Al anular por segunda vez la misma transferencia, la respuesta es 409 `CONFLICT` y los saldos no vuelven a cambiar.
- Dadas las transferencias sembradas en estado `pendiente` o `rechazada`, al anularlas la respuesta es 409 `CONFLICT`.
- Dada una transferencia interna cuyo destino quedó con menos saldo que el importe transferido, al anularla la respuesta es 409 `CONFLICT` y ningún saldo cambia.
- Al anular una transferencia inexistente, la respuesta es 404 `NOT_FOUND`.

RF-G4-TK — Obtener el token del grupo

- Con el usuario y la contraseña del grupo 4, la respuesta es 200, `data.tokenType` es `Bearer`, `data.expiresIn` es 3600 y `data.grupo` es 4.
- Una request posterior a cualquier endpoint del módulo enviando solo `Authorization: Bearer <token>` (sin `x-api-key`) responde con éxito.
- Con una contraseña incorrecta, la respuesta es 400 `VALIDATION_ERROR` con el mensaje "Usuario o password inválidos."; con un usuario inexistente, la respuesta es idéntica.
- Con las credenciales válidas del grupo 5 sobre esta ruta, la respuesta es 403 `FORBIDDEN`.
- Con un token alterado o vencido en el header `Authorization`, cualquier endpoint responde 401 `UNAUTHORIZED`.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Reglamentación del BCP sobre el Sistema de Pagos del Paraguay (SIPAP) y transferencias electrónicas	Toda orden de transferencia debe tener identificación unívoca, trazabilidad de origen y destino y un estado final de liquidación.	Parcial: cada transferencia tiene referencia única, deja movimientos trazables y queda `completada`; pero una transferencia externa no se liquida contra ningún otro banco, solo debita el origen.
Marco del BCP sobre disponibilidad de fondos en cuentas de depósito	Una orden solo puede ejecutarse si la cuenta ordenante está operativa y tiene fondos suficientes.	Cubierto: la transferencia exige origen `activa` y saldo suficiente, y es atómica.
Ley 1015/97 y debida diligencia de operaciones (SEPRELAD)	Registro de las operaciones con datos suficientes para su monitoreo, con umbrales y alertas por operaciones inusuales.	Parcial: queda registrado importe, cuentas, fecha y concepto, pero la API no tiene umbrales ni límites; el tope diario existe solo en la pantalla web.
Ley 1334/98 de Defensa del Consumidor (SEDECO), deber de información	El usuario debe recibir constancia de la operación con importe, destino, referencia y estado.	Cubierto: la API devuelve el comprobante completo y la web lo muestra al finalizar y en el detalle.
Reglas de conversión de moneda del BCP	Una transferencia entre cuentas de distinta moneda requiere un tipo de cambio explícito.	Cubierto por restricción: la API rechaza transferir entre cuentas de distinta moneda en lugar de convertir.
Buenas prácticas de reversa de operaciones	La reversa de una transferencia requiere causa justificada, autorización y, para el destino, su consentimiento o un proceso de reclamo.	No implementado: cualquier credencial puede anular cualquier transferencia completada, sin motivo obligatorio, sin plazo y retirando el dinero del destino sin intervención de su titular.

Brechas conocidas del sandbox

- Una transferencia externa debita el origen pero no acredita nada: el dinero sale del sandbox. No hay cuenta de compensación ni estado intermedio.
- Los estados `pendiente` y `rechazada` solo aparecen en los datos sembrados: la API crea toda transferencia como `completada` y no hay endpoint para confirmar o rechazar.
- La API no tiene límites por operación ni por día, ni control de duplicados: dos transferencias idénticas seguidas son aceptadas.
- No se valida que el beneficiario pertenezca al titular de la cuenta de origen: se puede transferir a un beneficiario agendado por otro cliente.
- La anulación no mira el estado actual de las cuentas: acredita o debita cuentas bloqueadas o cerradas, incluso una cuenta de destino dada de baja.
- La anulación no tiene plazo ni exige motivo, y puede dejar al destino sin el dinero sin que su titular intervenga.
- `GET /transferencias` no filtra por destino, beneficiario, cliente ni fecha: no se pueden listar por la API las transferencias recibidas por una cuenta.
- La unicidad de beneficiarios incluye a los dados de baja y no hay forma de reactivarlos: un número de cuenta que un cliente agendó y quitó no puede volver a agendarse.
- Banco, número de cuenta y alias del beneficiario son texto libre en la API; todo el formato lo impone solo la pantalla web.
- La referencia se genera con la hora al milisegundo y el id del origen: dos transferencias desde la misma cuenta en el mismo milisegundo colisionan y la segunda falla con 409.
- Las transferencias sembradas no son contablemente completas: varias en estado `completada` no tienen su movimiento en el extracto, por lo que anular una sembrada genera una reversa sin su asiento original.
- No existe control de titularidad: cualquier credencial del curso puede transferir desde la cuenta de cualquier cliente.